

StudyAI – Smart AI Study Companion & Quiz Generator

Project Description:

StudyAI is an AI-powered learning platform designed to help students study more efficiently using Generative AI. The application allows users to upload study materials such as PDF documents and notes, generate AI-powered summaries, create flashcards, prepare quizzes, and organize personalized study plans.

StudyAI integrates the Groq API with the LLaMA 3.3-70B Versatile model to generate accurate educational content in real time. Firebase Authentication provides secure user login while Firestore stores user information and study history. The platform also includes a local JSON storage fallback to ensure uninterrupted operation during development.

Built using React, Flask, Firebase, and Tailwind CSS, StudyAI provides a modern, responsive, and scalable learning experience for students.

Scenarios

Scenario 1

A student uploads a PDF containing Operating Systems notes.

StudyAI extracts the text, generates a concise AI summary, creates flashcards, and prepares a quiz to help the student revise quickly.

Scenario 2

A university student preparing for semester examinations uploads Data Structures notes.

StudyAI creates chapter-wise summaries, important questions, and multiple-choice quizzes to improve exam preparation.

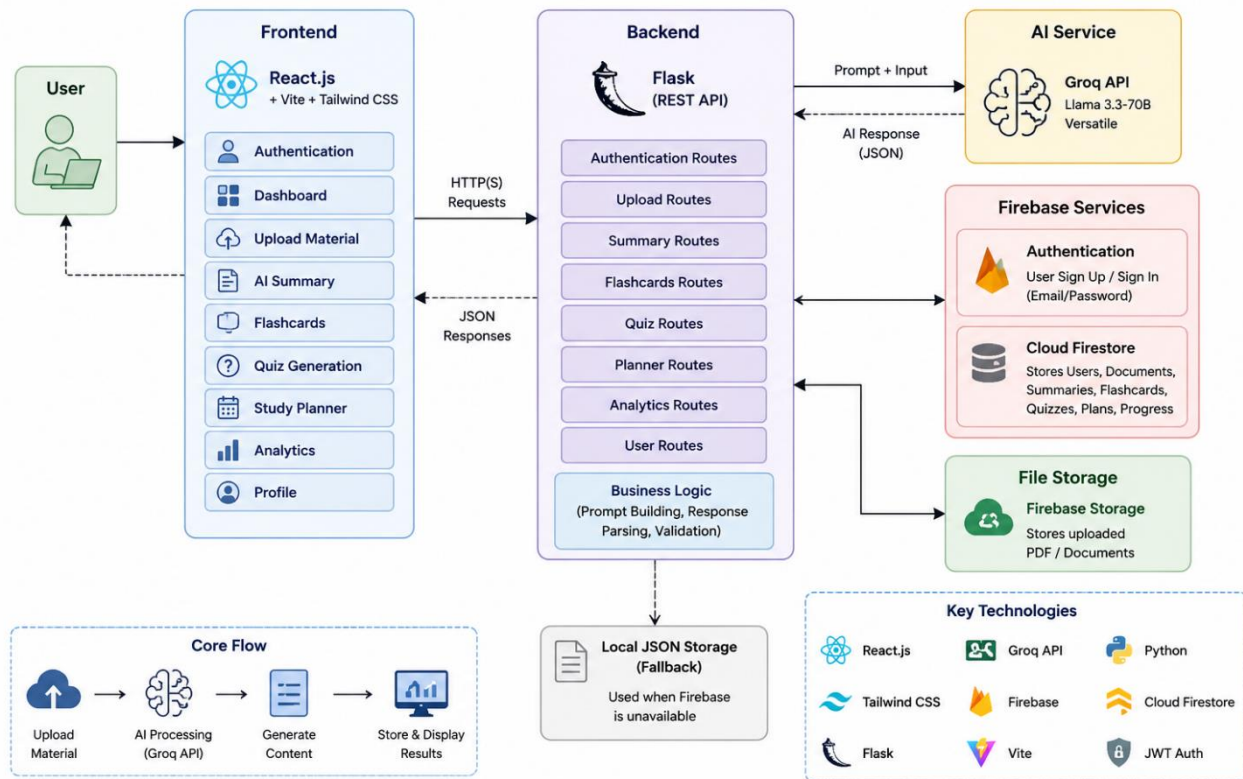
Scenario 3

A student revises Machine Learning concepts before an interview.

StudyAI generates flashcards, explains difficult topics, and tracks study progress through the dashboard.

Technical Architecture:

StudyAI – System Architecture

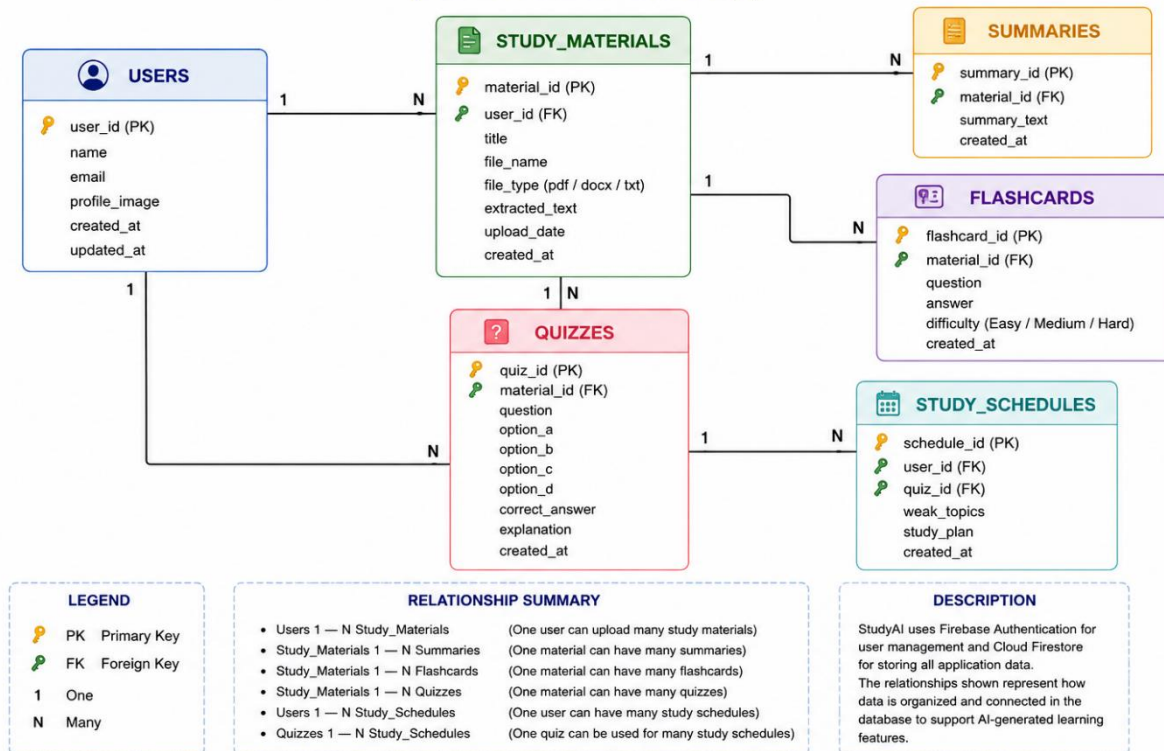


Description: StudyAI follows a modular three-tier architecture consisting of a React.js frontend, a Flask backend, and AI services powered by the Groq API. The frontend provides an intuitive interface where students can upload study materials and access AI-generated learning resources. The Flask backend handles authentication, file processing, prompt generation, and API communication. Firebase Authentication securely manages user login while Firestore stores user profiles and study history. When Firebase credentials are unavailable, the application automatically falls back to local JSON storage, ensuring uninterrupted functionality during development. The backend communicates with the Groq API using the LLaMA 3.3-70B Versatile model to generate summaries, flashcards, quizzes, and personalized study schedules.

ER Diagram:

StudyAI – ER Diagram

(Firestore Database Design)



ER Diagram Description

StudyAI uses Firebase Authentication and Cloud Firestore to manage users and AI-generated study resources.

Users

Stores authenticated user information such as User ID, name, email, profile image, and account creation date.

Study Materials

Contains uploaded study materials including PDFs, DOCX files, or text. Each material belongs to one user and stores extracted text used for AI processing.

Summaries

Stores AI-generated summaries created from uploaded study materials.

Flashcards

Stores AI-generated question-answer flashcards with difficulty levels for revision.

Quizzes

Stores AI-generated quizzes containing multiple-choice questions, options, correct answers, and explanations.

Study Schedules

Stores personalized AI-generated study plans created based on uploaded material and quiz performance.

Relationships

Entity 1	Relationship	Entity 2
User	One-to-Many	Study Materials
Study Material	One-to-Many	Summaries
Study Material	One-to-Many	Flashcards

Entity 1	Relationship	Entity 2
Study Material	One-to-Many	Quizzes
User	One-to-Many	Study Schedules
Quiz	One-to-Many	Study Schedules

Pre-requisites:

- Python
- Flask
- React
- Vite
- Firebase
- Firestore
- Groq API
- HTML
- CSS
- JavaScript
- Git
- Node.js
- npm

Project Workflow:**Milestone 1: Model Selection and Architecture**

In this milestone, the team focused on selecting the appropriate generative AI model from Groq for StudyAI's content-generation needs. This involved researching the capabilities and performance of available models and ensuring the chosen model aligned with the goal of generating structured summaries, flashcards, quizzes, and study schedules.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, a valid Groq API key is required. The following steps were followed to create one and connect it securely to the project:

- Step 1 - Create a Groq Account: Visit <https://console.groq.com> and sign up using a Google account or email address.
- Step 2 - Navigate to API Keys: After logging in, open the profile menu and select "API Keys", or go directly to the API Keys page.
- Step 3 - Create a New API Key: Click "Create API Key" and give it a descriptive name (e.g., studyai-key) so it is easy to identify later.
- Step 4 - Copy the API Key: Provide a Display Name, leave Expiration optional, and click Submit. Copy the key immediately, as it cannot be viewed again after the dialog closes.
- Step 5 - Add the Key to the Project: Create a .env file in the backend root and add:
GROQ_API_KEY=your_copied_api_key_here
- Step 6 - Verify the Key is Loaded: app.py loads the key automatically at startup using python-dotenv (load_dotenv()) before initializing the Groq client.
- Important - Never Commit the API Key: .env is added to .gitignore to prevent accidentally pushing the key to a public repository.

Activity 1.2: Research and Select the Appropriate Generative AI Model

- Understand the Project Requirements: Reviewed StudyAI's specific needs - generating summaries, flashcards, quizzes, and adaptive schedules from varying lengths of study material.
- Explore Groq's Model Documentation: Examined available LLM models, including their context windows, speed, and limitations.
- Evaluate Model Performance: Compared models on response quality, generation speed, and reliability of structured JSON output.
- Select the Optimal Model: Chose llama-3.3-70b-versatile for its balance of reasoning quality and generation speed, configured with a temperature of 0.5 and max_tokens of 3000 to allow for detailed, well-structured output across all four content types.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Created a visual representation of the application architecture, including the React.js frontend, Flask backend, Groq API integration, and Firebase data store (see Technical Architecture above).
- Detail Frontend Functionality: Outlined how users upload material, choose a feature (Summary, Flashcards, Quiz, or Schedule), and view AI-generated results on the dashboard.
- Outline Backend Responsibilities: Specified how the Flask backend handles incoming requests to /upload, /summarize, /flashcards, /quiz, and /schedule, validates and sanitizes input, builds structured prompts, and communicates with the Groq API.
- Describe AI Integration Points: Defined how the backend constructs prompts using a FEATURE_PROMPTS dictionary, calls the Groq API, parses the JSON response using extract_json(), and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- Install Python and Node.js: Ensured Python 3.8+ and Node.js 16+ (with npm) were installed on all team machines.
- Create a Backend Virtual Environment: Set up a virtual environment using venv to isolate backend dependencies.

```
D:\studyai\backend>python -m venv myenv  
D:\studyai\backend>"myenv\Scripts\activate"
```

- Install Backend Dependencies: Used pip to install Flask, the Groq SDK, python-dotenv, firebase-admin, PyPDF2, and python-docx.

```
(myenv) D:\studyai\backend> pip install Flask==3.0.3 groq==0.11.0 python-dotenv==1.0.1 firebase-admin PyPDF2 python-docx
```

- Configure the API Key: Created a .env file in the backend root and added the Groq API key and (optionally) Firebase service-account credentials.
- Initialize the Frontend: Scaffolded the React application using Create React App / Vite.

```
D:\studyai>npx create-react-app frontend  
D:\studyai\frontend>npm install axios react-router-dom
```

- Set Up Application Structure: Organized the project into backend/ (app.py, routes/, utils/, requirements.txt, .env) and frontend/ (src/components/, src/pages/, package.json).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implemented the four core AI generation features as a unified set of Groq-powered responses, each driven by a dedicated structured prompt:

- Summary Generator: Produces a structured markdown summary with a topic overview, key concepts, important definitions, core principles, and bullet-point revision notes.
- Flashcard Generator: Produces question-and-answer pairs with topic labels and difficulty levels (Easy / Medium / Hard).
- Quiz Generator: Produces MCQ, True/False, or Short Answer questions (configurable count) with correct answers and explanations.
- Adaptive Schedule Generator: Produces a day-by-day 7-day study plan with tasks, time slots, focus topics, and study tips, factoring in any weak topics identified from prior quiz results.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for material upload and each of the four AI-generation endpoints.

Process User Input:

- Accept multipart file uploads (PDF/DOCX) or pasted raw text through the React upload form.
- Extract text using PyPDF2 for PDFs and python-docx for Word files, then store the material with a unique ID.
- Validate that the extracted content is non-empty and within a reasonable character limit before processing.

Integrate API Calls:

- Within each route handler, call build_prompt() to construct a structured prompt combining the feature type and the material content.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

.Milestone 3: Backend (app.py) Development

This milestone focused on creating and refining the core logic of app.py, which serves as the backbone of the StudyAI application, establishing reliable prompt construction, API interaction, and data persistence.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes:

```
@app.route("/upload", methods=["POST"])
def upload_material():
    file = request.files.get("file")
    text_input = request.form.get("text", "")
    content = extract_text(file) if file else text_input
    material_id = store_material(content)
    return jsonify({"success": True, "material_id": material_id})
```

```
@app.route("/summarize/<material_id>", methods=["POST"])
def summarize(material_id):
    content = get_material(material_id)
    prompt = build_prompt("summary", content)
    response = client.chat.completions.create(
        model="llama-3.3-70b-versatile",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.5, max_tokens=3000,
    )
    result = extract_json(response.choices[0].message.content)
    return jsonify({"success": True, "summary": result})
```

Similar route handlers were implemented for /flashcards/<material_id>, /quiz/<material_id>, and /schedule/<material_id>, each using a feature-specific prompt template.

Define Prompt Engineering Helpers:

- FEATURE_PROMPTS dictionary: maps summary, flashcards, quiz, and schedule to detailed instruction templates that enforce strictly valid JSON output.
- build_prompt(feature, content): assembles the structured prompt string from the feature template and the material content.
- extract_json(text): a robust parser that first attempts direct json.loads(), then strips markdown code fences, and finally uses a regex to locate a raw JSON object, ensuring reliable parsing across model response variations.
- identify_weak_topics(quiz_results): aggregates incorrect quiz answers by topic and feeds them into the schedule prompt for adaptive scheduling.

Milestone 4: Frontend Development

This milestone focused on developing a clean, student-friendly interface for StudyAI, using React.js components for the upload flow, summary view, flashcard deck, quiz panel, and schedule timeline.

Activity 4.1: Designing and Developing the User Interface

- Build the React dashboard as the main application shell with navigation to Upload, Summary, Flashcards, Quiz, and Schedule pages.
- Use a component-based structure: UploadCard, SummaryView, FlashcardDeck, QuizPanel, ScheduleView, and AnalyticsChart.
- Apply a clean, student-friendly design with clear typography, color-coded topic tags, and responsive layout for mobile and desktop.

Activity 4.2: Creating Dynamic Content Rendering with React

- Handle file/text submission asynchronously using Axios calls from the upload form to the Flask backend's /upload endpoint.
- Show a loading spinner while awaiting AI responses from the /summarize, /flashcards, /quiz, and /schedule endpoints.
- Render summaries as formatted markdown, flashcards as flippable cards with shuffle and mastery tracking, quizzes as step-through question panels with immediate scoring and explanations, and schedules as a day-by-day timeline.

- Update the Analytics dashboard after each quiz submission with a weak-topic breakdown pulled from the backend.

Milestone 5: Deployment

This milestone focused on deploying StudyAI locally to verify that all features function correctly and preparing the application for demonstration.

Activity 5.1: Preparing the Application for Local Deployment

- Set up and activate the backend virtual environment.
- Install all backend dependencies using requirements.txt.
- Install frontend dependencies using npm install.
- Configure environment variables in the .env file, including the Groq API key and Firebase credentials.
- Ensure all required Python and Node.js packages are installed before running the application.

Activity 5.2: Local Testing and Verification

Run the Flask Backend

cd backend

python app.py

Expected Output:

* Running on http://127.0.0.1:5000

```

PS C:\full stack projects\github folder\Study_AI-platform\backend> python app.py
[2026-07-13 21:35:17] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\uploads
[2026-07-13 21:35:17] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\storage
[2026-07-13 21:35:17] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\logs
[2026-07-13 21:35:17] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\uploads\profile_photos
[2026-07-13 21:35:17] INFO studyai.app - CORS enabled for all origins
[2026-07-13 21:35:17] DEBUG config - .env loaded from: C:\full stack projects\github folder\Study_AI-platform\backend\.env
[2026-07-13 21:35:17] DEBUG config - Current working directory: C:\full stack projects\github folder\Study_AI-platform\backend
[2026-07-13 21:35:17] DEBUG config - FIREBASE_PROJECT_ID set: True
[2026-07-13 21:35:17] DEBUG config - FIREBASE_CLIENT_EMAIL set: True
[2026-07-13 21:35:17] DEBUG config - FIREBASE_PRIVATE_KEY set: True
[2026-07-13 21:35:17] DEBUG config - FIREBASE_PRIVATE_KEY length: 1704
[2026-07-13 21:35:17] INFO firebase.firebase_config - 🔥 Firebase initialized successfully - Firestore ACTIVE
[2026-07-13 21:35:17] INFO firebase.firebase_config - 🔥 Firestore is ready (collections will auto-create)
[2026-07-13 21:35:17] INFO studyai.app - 🔥 Firebase initialized successfully
[2026-07-13 21:35:17] INFO services.local_storage - ✅ Local JSON storage initialised at: C:\full stack projects\github folder\Study_AI-platform\backend\storage
[2026-07-13 21:35:17] INFO studyai.app - 📁 Local storage initialized
[2026-07-13 21:35:19] INFO services.groq_service - ✅ Groq client initialised - model: llama-3.3-70b-versatile, max_tokens: 4096
[2026-07-13 21:35:19] INFO services.groq_service - ✅ Groq client initialised - model: llama-3.3-70b-versatile, max_tokens: 4096
[2026-07-13 21:35:19] INFO services.groq_service - ✅ Groq client initialised - model: llama-3.3-70b-versatile, max_tokens: 4096
[2026-07-13 21:35:20] INFO services.groq_service - ✅ Groq client initialised - model: llama-3.3-70b-versatile, max_tokens: 4096
[2026-07-13 21:35:20] INFO services.groq_service - ✅ Groq client initialised - model: llama-3.3-70b-versatile, max_tokens: 4096
[2026-07-13 21:35:20] INFO studyai.app - Blueprints registered successfully
[2026-07-13 21:35:20] INFO studyai.app - StudyAI Backend v1.0.0 ready - env: development | 0.0.0.0:5000
[2026-07-13 21:35:20] INFO studyai.app - ✅ All required configuration is present.
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.37.202.60:5000
Press CTRL+C to quit
* Restarting with stat
[2026-07-13 21:35:21] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\uploads
[2026-07-13 21:35:21] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\storage
[2026-07-13 21:35:21] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\logs
[2026-07-13 21:35:21] INFO config - Checked directory: C:\full stack projects\github folder\Study_AI-platform\backend\uploads\pr
  
```

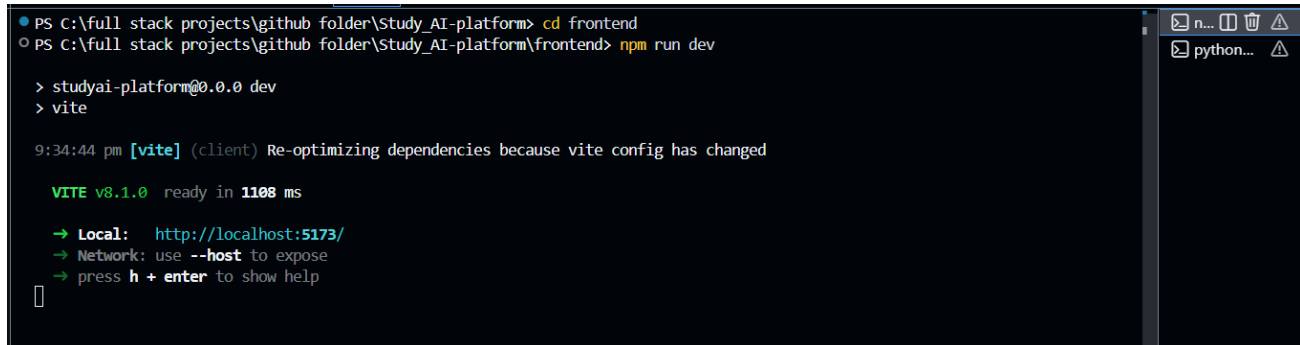

Run the React Frontend

```
cd frontend
```

```
npm run dev
```

Expected Output:

Local: <http://localhost:5173>



```
PS C:\full_stack_projects\github_folder\Study_AI-platform> cd frontend
PS C:\full_stack_projects\github_folder\Study_AI-platform\frontend> npm run dev

> studyai-platform@0.0.0 dev
> vite

9:34:44 pm [vite] (client) Re-optimizing dependencies because vite config has changed

VITE v8.1.0 ready in 1108 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

This milestone focused on deploying StudyAI first locally to verify correct operation, and then preparing it for a shared demo environment.

Activity 5.1: Preparing the Application for Local Deployment

- Set up and activate the backend virtual environment, then install all dependencies from requirements.txt.
- Install frontend dependencies with npm install inside the frontend/ directory.
- Configure environment variables for the Groq API key and, if available, Firebase service-account credentials.

Activity 5.2: Local Testing and Verification

- Run the Flask backend locally:

```
(myenv) D:\studyai\backend>python app.py
* Running on http://127.0.0.1:5000
```

- Run the React development server in a separate terminal:

```
D:\studyai\frontend>npm start
Compiled successfully! Local: http://localhost:3000
```

- Test all four AI features end-to-end (Summary, Flashcards, Quiz, Schedule) using a sample study document to verify correct generation, parsing, and rendering.

Activity 5.3: Deployment for Demonstration

- The StudyAI application was successfully demonstrated in the local development environment. All major functionalities—including user authentication, study material upload, AI-generated summaries, flashcards, quizzes, personalized study schedules, and analytics—worked as expected. The application successfully communicated with the Groq API to generate educational content in real time while securely managing user data through Firebase Authentication and Cloud Firestore.

StudyAI

[Features](#) [How it works](#) [Pricing](#) | [Log in](#) [Sign up](#)

Introducing StudyAI 2.0

Learn faster with Intelligent Study Tools

Upload your materials and instantly get AI-generated
summaries, adaptive flashcards, and personalized quizzes.

[Start Studying Free →](#)[See How It Works](#)

StudyAI

[Features](#) [How it works](#) [Pricing](#) | [Log in](#) [Sign up](#)

Welcome Back

Enter your details to access your dashboard

Email Address

Password

[Forgot password?](#)[Sign In →](#)

Don't have an account? [Register](#)

StudyAI



MAIN MENU

[Overview](#)[Upload Materials](#)[AI Summaries](#)[Flashcards](#)[Quiz Center](#)[Study Planner](#)[Analytics](#)

ACCOUNT

 **uday mallela**
Profile Settings**Welcome back, uday mallela! 🙌**

Here's a summary of your study progress.

 Total Materials
23 Quizzes Taken
10 Study Streak
9 days Avg. Score
53%

Recent Materials

- | | | |
|---|---|------------------------|
|  | Mallela Udaykumar Reddy
9 days ago | Review |
|  | KARRI VARUN SRI TEJA
6 days ago | Review |
|  | The Failure of Classical
16 days ago | Review |

Weak Topics to Review

- | | |
|---|--------------------------|
|  Pauli Exclusion Principle | Practice |
|  Threshold Frequency | Practice |
|  Stopping Potential | Practice |



StudyAI





MAIN MENU

- Overview
- Upload Materials**
- AI Summaries
- Flashcards
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

 **uday mallela**
Profile Settings

Upload Study Material

Upload PDFs, DOCX, or TXT files for AI analysis.



Filled_Empathy_Map.pdf
114.2 KB

 **Upload complete!**

Title: Filled_Empathy_Map.pdf
Words: 0
Pages: 0

What would you like to create?





Upload Another File



StudyAI





MAIN MENU

- Overview
- Upload Materials
- AI Summaries**
- Flashcards
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

 **uday mallela**
Profile Settings

Empathy Map (PDF)



[← Back to Summaries List](#)

 **Empathy Map** 

Overview

The empathy map is a tool used to understand the thoughts, feelings, and behaviors of the primary user, college students preparing for exams, to design a customer-centric solution. It highlights the challenges students face, such as managing study materials and reducing revision time. The empathy map guides the development of StudyAI, a smart AI study companion and quiz generator, to address these challenges.

Key Concepts

Empathy Map: A visual tool used to understand the thoughts, feelings, and behaviors of a target user

User-Centric Design: A design approach that focuses on the needs and wants of the target user

StudyAI: A smart AI study companion and quiz generator designed to help college students prepare for exams

Revision Note: Identify the target user and their challenges. Create an empathy map to guide the design of a solution

Your Summaries (13) 



1 GUMMIDI VENKATA SAI GANESH
6 Jul 2024

[View →](#)



The Failure of Classical
27 Jun 2024

[View →](#)



StudyAI





MAIN MENU

- Overview
- Upload Materials
- AI Summaries
- Flashcards**
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

 **uday mallela**
Profile Settings

[← Back to Flashcard Sets](#)

Flashcard Generator

Master your topics with interactive flashcards.

Card 1 of 20

1 mastered 

INTRODUCTION

EASY

What is Mallela Udaykumar Reddy's current field of study?



[←](#)  [→](#)

StudyAI

MAIN MENU

- Overview
- Upload Materials
- AI Summaries
- Flashcards
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

uday mallick

StudyAI

Back to Quizzes

Mallela Udaykumar Reddy

Question 1 of 10

9:58

medium

MCQ

What is Mallela Udaykumar Reddy's current level of education?

A) B.Tech in Computer Science (Completed)

B) B.Tech in Computer Science (Ongoing)

C) M.Tech in Computer Science (Ongoing)

D) Ph.D. in Computer Science (Ongoing)

Previous

Next

MAIN MENU

- Overview
- Upload Materials
- AI Summaries
- Flashcards
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

uday mallick

StudyAI

Study Planner

Your AI-generated daily study schedule.

Export PDF Schedule

Day 1: Jul 9, 2026

Review limits and basic properties of limits

Practice solving limit problems using direct substitution and factoring

Day 2: Jul 10, 2026

Study derivatives and their geometric interpretation

Practice finding derivatives of polynomial and rational functions

Day 3: Jul 11, 2026

Learn the power rule, product rule, and quotient rule for differentiation

Apply these rules to solve differentiation problems

Day 4: Jul 12, 2026

Focus on applications of derivatives, including optimization and motion along a line

Solve problems related to these applications

Day 5: Jul 13, 2026

PLAN PROGRESS

0%

0 of 7 days completed

PLANNER OPTIONS

Generate New Plan

MAIN MENU

- Overview
- Upload Materials
- AI Summaries
- Flashcards
- Quiz Center
- Study Planner
- Analytics

ACCOUNT

uday mallick

StudyAI

Analytics Dashboard

Track your learning progress over time.

Keep practicing — your current average is 52%. Focus on Photon Energy.

Documents Uploaded

24

Quizzes Taken

10

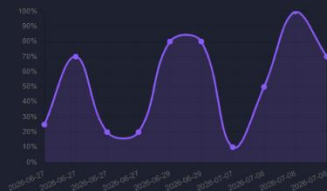
Average Score

52.5%

Study Streak

0 days

Quiz Performance Trend



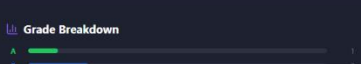
Weak Topic Mastery



Score Distribution



Grade Breakdown



.Milestone 6: Conclusion

Conclusion:

StudyAI is a generative AI platform built with React.js, Flask, and Firebase that streamlines exam preparation by turning raw study material into summaries, flashcards, quizzes, and adaptive schedules. It uses Groq's high-speed inference API with the Llama 3.3-70B Versatile model to instantly generate structured, tailored content, while gracefully falling back to local JSON storage when Firebase credentials are unavailable. The project - spanning model selection, backend and frontend development, and local/demo deployment - illustrates how targeted AI integration and a structured framework can meaningfully support personalized learning.

Looking ahead, StudyAI offers clear opportunities for expansion, including multi-language content generation, collaborative study groups, voice-based quiz interaction, and full user authentication to support saved histories across devices. By continuing to refine the underlying prompts and expanding the analytics dashboard, the platform is well-positioned to grow from a college project into a comprehensive personalized learning assist.